Математички методи за машинско учење 2025/2026

Домаћи задатак број 5

Име и презиме студента: Jovan Paunović

Број индекса: 2122

Упутство за израду и предају домаћег задатака:

1. Пре почетка израде промените име датотеке у 05Domaci_Ime_Prezime. (убаците своје име и презиме)
2. Попуните ћелију испод наслова одговарајућим подацима.
1. Употреба ћирилице није обавезна за предају домаћег задатка.
4. За решавање задатака, уколико је потребно, отворите испод текста задатка додатне ћелије за уписивање текстуалног одговора (Markdown) или програмског кода (Code).
1. Сва израчунавања, уколико је потребно, вршити у Python-у.
5. Након завршетка израде решења домаћег Notebook документ сачувати у pdf формату и проследити га наставнику. То можете да урадите или кроз Teams или на мејл адресу jovana.dzunic@elfak.ni.ac.rs

In [59]:
```
import numpy as np
import numpy.random as rndm
import matplotlib.pyplot as plt
from scipy.linalg import svd
from scipy.linalg import toeplitz
```

Задатак 1. У многим AI алгоритмима користимо kernel матрицу или Грамову матрицу да бисмо мерили сличност између података. Да би алгоритам радио коректно, ова матрица мора бити позитивно дефинитна.

а) Нека је дат скуп вектора (података)

$$x_1, x_2, \dots, x_n \in \mathbb{R}^d,$$

смештених дуж врста матрице X. Дефинишимо Грамову матрицу

$$G \in \mathbb{R}^{n \times n}$$

тако да је

$$G_{ij} = x_i^T x_j.$$

Доказати да је Грамова матрица

$$G = XX^T$$

увек позитивно семидефинитна.

(5 поена)

Matrica G je pozitivno semidefinitna ako za svaki vektor $v \in \mathbb{R}^n$ važi

$$v^T G v \geq 0.$$

Za proizvoljan vektor $v \in \mathbb{R}^n$ možemo da napišemo:

$$v^T G v = v^T (XX^T) v.$$

Zbog asocijativnosti množenja matrica važi:

$$v^T (XX^T) v = (v^T X)(X^T v).$$

Zatim primećujemo da je

$$v^T X = (X^T v)^T.$$

Dakle:

$$(v^T X)(X^T v) = (X^T v)^T (X^T v).$$

Ovo je skalarni proizvod vektora sa samim sobom:

$$(X^T v)^T (X^T v) = \|X^T v\|^2 \geq 0.$$

Dakle, pošto je $\|X^T v\|^2 \geq 0$ za svaki $v \in \mathbb{R}^n$, sledi da je

$$v^T G v \geq 0, \quad \forall v \in \mathbb{R}^n.$$

Dakle, Gramova matrica $G = XX^T$ je uvek pozitivno semidefinitna.

5/5поена

б) Ако Грамовој матрици G додамо члан λI (где је $\lambda > 0$), доказати да резултат постаје строго позитивно дефинитна матрица.
Ово се у AI литератури назива **регуларизација Тихонова** или **Ridge**.

(5 поена)

Matrica A je pozitivno definitna ako za svaki nenula vektor $v \in \mathbb{R}^n$ važi

$$v^T A v > 0.$$

Posmatrajmo matricu

$$G_\lambda = G + \lambda I, \quad \lambda > 0.$$

Za proizvoljan vektor $v \in \mathbb{R}^n$ imamo:

$$v^T G_\lambda v = v^T (G + \lambda I) v.$$

Razdvajamo:

$$v^T (G + \lambda I) v = v^T G v + \lambda v^T I v.$$

Znamo da je $v^T G v \geq 0$ (jer je G pozitivno semidefinitna).
Takođe,

$$v^T I v = v^T v = \|v\|^2.$$

Dakle:

$$v^T G_\lambda v = v^T G v + \lambda \|v\|^2.$$

Pošto za svaki nenulti vektor v važi $\|v\|^2 > 0$ i $\lambda > 0$ i $v^T G v \geq 0$ dobijamo

$$v^T G v + \lambda \|v\|^2 > 0,$$

odnosno

$$v^T G_\lambda v > 0$$

pa je matrica G_λ pozitivno definitna.

5/5поена

Задатак 2. Инверзни проблеми

Инверзни проблеми (или повратни проблеми) су математички проблеми у којима, уместо да израчунамо резултат на основу улазних података (као код директних проблема), ми тражимо узрок, унутрашње карактеристике или параметре система на основу познатих мерења, података или ефеката на излазу.

То је процес враћања уназад - од онога што видимо (резултат) до онога што је то узроковало (параметре).

Директан проблем: Знамо модел K и улаз x, израчунавамо излаз $y = Kx$.

Другим речима, знамо узрок (нпр. облик објекта, материјал) и модел како се он понаша. Израчунавамо резултат (нпр. како објекат изгледа на слици, како се шири топлота).

Инверзни проблем: Знамо излаз y и модел K, тражимо улаз $x = K^{-1} y$.

Знамо резултат (нпр. слику) и модел.
Циљ је да пронађемо узрок (нпр. прави облик објекта, непознате карактеристике материјала).

Особине инверзних проблема

За разлику од директних проблема који су обично добро постављени (постоји јединствено решење), инверзни проблеми често **немају јединствено решење**, или су решења **веома осетљива на грешке у подацима** (лоше условљени).

Жак Адамар је дефинисао да је инверзни проблем **коректно постављен (well-posed)** ако су испуњени следећи услови:

1. **Решење постоји.**
2. **Решење је јединствено.**
3. **Решење је стабилно** или добро условљено (непрекидно зависи од података y).

Ако било који од ових услова није испуњен (што је најчешћи случај у вештачкој интелигенцији и обради сигнала), кажемо да је проблем **некоректно постављен (ill-posed)** према Адамару.

a) Нека је $K \in M_{n\times n}$ симетрична, позитивно дефинитна матрица са сопственим вредностима

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n > 0$$

и одговарајућим ортонормираним сопственим векторима

$$q_1, \ldots, q_n.$$

Спектрална декомпозиција матрице гласи:

$$K = Q\Lambda Q^T,$$

где је $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$.

Показати да се решење x инверзног проблема може записати у облику

$$x = \sum_{i=1}^{n} \frac{1}{\lambda_i}(q_i^T y)\, q_i.$$

Коју врсту трансформације вектора y представља израз $(q_i^T y)\, q_i$?

(5 поена)

Polazimo od osnovne jednačine direktnog problema:

$$y = Kx$$

Treba naći

$$x = K^{-1}y$$

Znamo da je spektralna dekompozicija matrice K data kao:

$$K = Q\Lambda Q^T$$

gde je Q ortogonalna matrica čije su kolone ortonormirani sopstveni vektori $q_1, \ldots, q_n$, a Λ je dijagonalna matrica sa sopstvenim vrednostima na dijagonali $\mathrm{diag}(\lambda_1, \ldots, \lambda_n)$.

Pošto je matrica Q ortogonalna, važi da je njena inverzna matrica jednaka njenoj transponovanoj matrici: $Q^{-1} = Q^T$. Takođe, inverzna matrica dijagonalne matrice Λ je jednostavno dijagonalna matrica čiji su elementi recipročne vrednosti originalnih: $\Lambda^{-1} = \mathrm{diag}(\frac{1}{\lambda_1}, \ldots, \frac{1}{\lambda_n})$.

Sada možemo da nađemo inverz matrice K (inverzna matrica proizvoda je proizvod pojedinačnih inverznih matrica, ali u suprotnom redosledu):

$$K^{-1} = (Q\Lambda Q^T)^{-1} = (Q^T)^{-1}\Lambda^{-1}Q^{-1} = Q\Lambda^{-1}Q^T$$

Zamenimo ovaj izraz u jednačinu za x:

$$x = Q\Lambda^{-1}Q^T y$$

Da bismo ovo zapisali u obliku sume, možemo da razložimo množenje matrica. Vektor y se prvo množi sa Q^T. Pošto su redovi matrice Q^T zapravo transponovani vektori q_i^T, rezultat $Q^T y$ je vektor-kolona čiji su elementi skalarni proizvodi $q_i^T y$:

$$Q^T y = \begin{bmatrix} q_1^T y \\ q_2^T y \\ \vdots \\ q_n^T y \end{bmatrix}$$

Zatim se ovaj vektor množi dijagonalnom matricom Λ^{-1}, što znači da se svaki i-ti element vektora množi sa $\frac{1}{\lambda_i}$:

$$\Lambda^{-1} Q^T y = \begin{bmatrix} \frac{1}{\lambda_1} q_1^T y \\ \frac{1}{\lambda_2} q_2^T y \\ \vdots \\ \frac{1}{\lambda_n} q_n^T y \end{bmatrix}$$

Na kraju, ovaj rezultat množimo sa matricom Q. Množenje matrice Q vektorom-kolonom je ekvivalentno linearnoj kombinaciji kolona matrice Q (a to su vektori q_i), gde su koeficijenti elementi tog vektora-kolone. Prema tome, dobijamo:

$$x = \sum_{i=1}^{n} \left(\frac{1}{\lambda_i} q_i^T y \right) q_i = \sum_{i=1}^{n} \frac{1}{\lambda_i} (q_i^T y) q_i$$

Izraz $(q_i^T y) q_i$ predstavlja ortogonalnu projekciju vektora y na pravac koji je određen sopstvenim vektorom q_i.

Ako posmatramo detaljnije:

- $q_i^T y$ predstavlja standardni skalarni proizvod. Pošto je q_i jedinični vektor (deo ortonormirane baze), ovaj skalarni proizvod predstavlja dužinu projekcije vektora y na pravac vektora q_i.
- $(q_i^T y) q_i$ - kada tu dužinu pomnožimo sa samim jediničnim vektorom q_i, dobijamo novi vektor istog pravca kao vektor q_i i čija je dužina tačno jednaka projekciji.

Dakle, cela formula za x praktično znači da vektor y projektujemo na svaki od pravaca iz baze sopstvenih vektora, svaku projekciju skaliramo faktorom $\frac{1}{\lambda_i}$, i na kraju ih sve saberemo.

Na osnovu toga možemo da vidimo potencijalni problem - ako postoje veoma male sopstvene

5/5поена

б) Када уместо y посматрамо зашумљене податке

$$y_\varepsilon = y + \varepsilon,$$

решење инверзног проблема постаје

$$x_\varepsilon = K^{-1} y_\varepsilon = K^{-1}(y + \varepsilon) = x + K^{-1}\varepsilon.$$

Записати $K^{-1}\varepsilon$ по угледу на формулу из дела под а). Нека је сопствена вредност $\lambda_n \approx 0$ блиска нули. Шта се дешава са решењем када се подацима y дода чак и минималан шум ε у оваквим околностима? Сопствене вредности $\frac{1}{\lambda_i}$ инверзне матрице K^{-1} посматрати као појачавач шума. Објаснити зашто ово крши Адамаров трећи услов о стабилности.

(5 поена)

Primenom potpuno iste formule iz dela pod (a) na vektor šuma ε, dobijamo:

$$K^{-1}\varepsilon = \sum_{i=1}^{n} \frac{1}{\lambda_i}(q_i^T \varepsilon) q_i$$

Zamenom ovoga u jednačinu za x_ε, ukupno rešenje inverznog problema sa zašumljenim podacima postaje:

$$x_\varepsilon = x + \sum_{i=1}^{n} \frac{1}{\lambda_i}(q_i^T \varepsilon) q_i$$

Sopstvene vrednosti inverzne matrice, $\frac{1}{\lambda_i}$, u gornjoj sumi direktno se ponašaju kao pojačavači šuma.

Kada je neka sopstvena vrednost izuzetno mala (npr. $\lambda_n \approx 0$), njena recipročna vrednost $\frac{1}{\lambda_n}$ postaje ekstremno velika.

Čak i ako podacima dodamo minimalan, jedva primetan šum ε, njegova projekcija na pravac vektora q_n (odnosno vrednost $q_n^T \varepsilon$) uglavnom neće biti tačno jednaka nuli. Kada se ova mala komponenta šuma pomnoži sa ogromnim faktorom pojačanja $\frac{1}{\lambda_n}$, greška "eksplodira".

Kao rezultat, član sabirka koji potiče od šuma:

$$\frac{1}{\lambda_n}(q_n^T \varepsilon) q_n$$

postaje dominantan, potpuno maskirajući pravo, korisno rešenje x. Krajnji rezultat x_ε imaće besmisleno velike vrednosti i gubi svoju upotrebljivost.

Adamarov treći uslov definiše da rešenje mora biti stabilno, što matematički znači da ono mora neprekidno zavisiti od podataka y. Drugim rečima, male promene (greške/šum) u podacima (y) smeju da izazovu samo srazmerno male promene u rešenju x.

Međutim, ono što vidimo je da kada postoji $\lambda_n \approx 0$, veoma mala promena na ulazu (dodavanje šuma ε) izaziva ogromnu promenu na izlazu (ogroman vektor greške $K^{-1}\varepsilon$).

Pošto mala greška u merenju rezultuje potpuno pogrešnim rešenjem, uslov neprekidne zavisnosti nije ispunjen. Rešenje je nestabilno (loše uslovljeno), odnosno kažemo da je prema Adamaru ovaj inverzni problem nekorektno postavljen.

5/5поена

в) Да бисмо решили проблем нестабилности и обезбедили трећи Адамаров услов, користимо **Тихоновљеву регуларизацију** у односу на величину проблема:

$$(K + n\lambda I)x = y, \qquad \lambda > 0.$$

Тада важи:

$$K + n\lambda I = Q(\Lambda + n\lambda I)Q^T,$$
$$(K + n\lambda I)^{-1} = Q(\Lambda + n\lambda I)^{-1}Q^T.$$

Регуларизовано решење може да се запише у облику:

$$x_\lambda = \sum_{i=1}^{n} \frac{1}{\lambda_i + n\lambda} (q_i^T y)\, q_i.$$

Описати на који начин фактор регуларизације утиче на фактор појачања шума.

(5 поена)

U neregularizovanom slučaju faktor pojačanja šuma je

$$\frac{1}{\lambda_i}$$

U regularizovanom slučaju faktor postaje

$$\frac{1}{\lambda_i + n\lambda}$$

Kada je λ_i veoma mala vrednost (npr. $\lambda_i \approx 0$), neregularizovani faktor $\frac{1}{\lambda_i}$ je ogroman i šum se snažno pojačava.

Dodavanjem $n\lambda$ u imenilac, ako se desi da je λ_i veoma malo (blisko nuli), faktor pojačanja ostaje ograničen:

$$\frac{1}{\lambda_i + n\lambda} \leq \frac{1}{n\lambda}$$

Dakle, kad je sopstvena vrednost velika, onda vrednost $n\lambda$ ne utiče mnogo na rešenje, a ako je sopstvena vrednost bliska nuli, onda dodavanje $n\lambda$ na sopstvenu vrednost znatno pomaže da količnik

$$\frac{1}{\lambda_i + n\lambda}$$

ne "eksplodira".

Time se sprečava nekontrolisano pojačavanje šuma i obezbeđuje stabilnost rešenja.

Na taj način se ispunjava treći Adamarov uslov o stabilnosti i problem postaje korektno postavljen (well-posed).

5/5поена

г) У овом делу ћемо практично видети како регуларизација спречава "експлозију" шума која је карактеристична за некоректне проблеме. нПокренути код у наставку за креирање елемената инверзног проблема.

In [60]:
```
# 1. Формирање лоше условљене матрице K (димензије 50x50)
n = 50
A = np.random.randn(n, n)
U, S, Vt = np.linalg.svd(A)

# Вештачки постављамо сопствене вредности тако да експоненцијално опадај
# Ово моделује типичан инверзни проблем који је некоректно постављен
S = np.exp(-0.5 * np.arange(n))
K = U @ np.diag(S) @ U.T

# 2. Правимо стварно решење 'x_true' и мерења 'y' са додатком малог шума
x_true = np.random.randn(n)
noise_level = 0.001
y_noisy = K @ x_true + noise_level * np.random.randn(n)
```

ВАШ ЗАДАТАК:

1. Израчунати наивно решење `x_naive = np.linalg.inv(K) @ y_noisy`
2. Израчунати регуларизовано решење `x_reg` за `lmbda = 0.01` користећи формулу са спектралним филтрирањем (преко сопствених вредности)
3. Израчунати Еуклидску грешку за оба решења у односу на `x_true`.
4. Визуелизовати `x_true`, `x_naive` и `x_reg` на једном графику како бисте видели разлику.

(10 поена)

In [61]:

```
x_naive = np.linalg.inv(K) @ y_noisy

lmbda = 0.01

# koristimo U i S iz konstrukcije matrice K
# jer znamo da je K = U @ diag(S) @ U^T
# pa su U i S upravo sopstveni vektori i vrednosti od K
x_reg = np.zeros(n)
for i in range(n):
    lambda_i = S[i]
    q_i = U[:, i]
    faktor = 1.0 / (lambda_i + n * lmbda)
    projekcija = np.dot(q_i, y_noisy)
    x_reg += faktor * projekcija * q_i

error_naive = np.linalg.norm(x_naive - x_true)
error_reg = np.linalg.norm(x_reg - x_true)

print(f"Euklidska greška naivnog rešenja: {error_naive:.3f}")
print(f"Euklidska greška regularizovanog rešenja: {error_reg:.3f}")

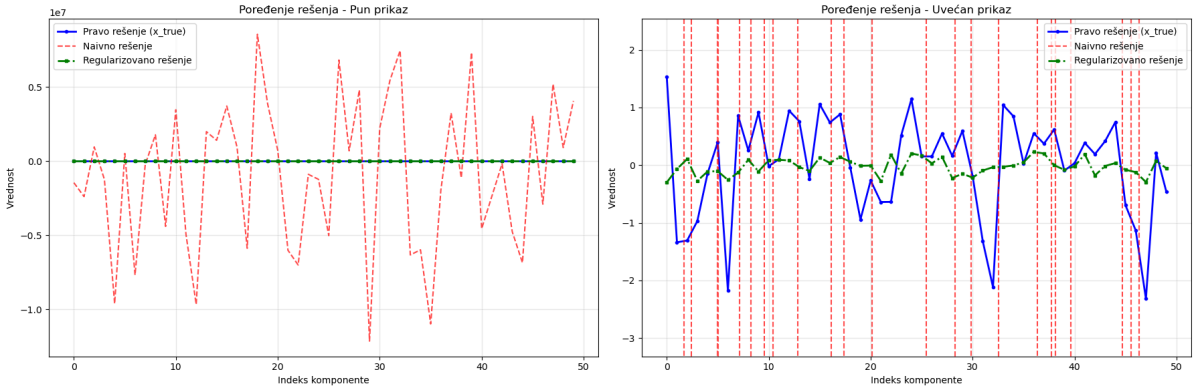
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))

# grafik 1: pokazuje punu "eksploziju" šuma
ax1.plot(x_true, 'b-', linewidth=2, label='Pravo rešenje (x_true)', mark
ax1.plot(x_naive, 'r--', linewidth=1.5, label='Naivno rešenje', alpha=0.
ax1.plot(x_reg, 'g-.', linewidth=2, label='Regularizovano rešenje', mark
ax1.set_xlabel('Indeks komponente')
ax1.set_ylabel('Vrednost')
ax1.set_title('Poređenje rešenja - Pun prikaz')
ax1.legend()
ax1.grid(True, alpha=0.3)

# grafik 2: uvećan prikaz (fokus na x_true i x_reg)
ax2.plot(x_true, 'b-', linewidth=2, label='Pravo rešenje (x_true)', mark
ax2.plot(x_naive, 'r--', linewidth=1.5, label='Naivno rešenje', alpha=0.
ax2.plot(x_reg, 'g-.', linewidth=2, label='Regularizovano rešenje', mark
ax2.set_xlabel('Indeks komponente')
ax2.set_ylabel('Vrednost')
ax2.set_title('Poređenje rešenja - Uvećan prikaz')
ax2.legend()
ax2.grid(True, alpha=0.3)
y_min = min(x_true.min(), x_reg.min()) - 1
y_max = max(x_true.max(), x_reg.max()) + 1
ax2.set_ylim(y_min, y_max)

plt.tight_layout()
plt.show()
```

```
Euklidska greška naivnog rešenja: 36076296.401
Euklidska greška regularizovanog rešenja: 5.963
```



10/10поена

д) Када матрица K није инвертибилна (сингуларна је), уместо K^{-1} користимо Мур–Пенроузов псеудоинверз K^{+}. Решење

$$x_{MP} = K^{+} y$$

је решење које минимизује норму остатка $\|Kx - y\|_2$, а међу свим таквим решењима бира оно са најмањом нормом $\|x\|_2$.

ВАШ ЗАДАТАК:

1. Израчунати решење преко Мур-Пенроузовог псеудоинверза: `x_mp = np.linalg.pinv(K) @ y_noisy`
2. Израчунати решење преко Тихоновљеве регуларизације: `lmbda = 0.01`, `x_reg = np.linalg.solve(K + n*lmbda*np.eye(n), y_noisy)`
3. Упоредити грешке: `err_mp = np.linalg.norm(x_mp - x_true)`, `err_reg = np.linalg.norm(x_reg - x_true)`
4. Нацртати графике и објаснити зашто `x_mp` "експлодира" (има велике вредности), док је `c_reg` глатко и близу `c_true`.

(10 поена)

In [62]:

```
x_mp = np.linalg.pinv(K) @ y_noisy

lmbda = 0.01
x_reg = np.linalg.solve(K + n * lmbda * np.eye(n), y_noisy)

err_mp = np.linalg.norm(x_mp - x_true)
err_reg = np.linalg.norm(x_reg - x_true)

print(f"Euklidska greška Mur-Penrouzovog pseudoinverza: {err_mp:.3f}")
print(f"Euklidska greška Tihonovljeve regularizacije: {err_reg:.3f}")

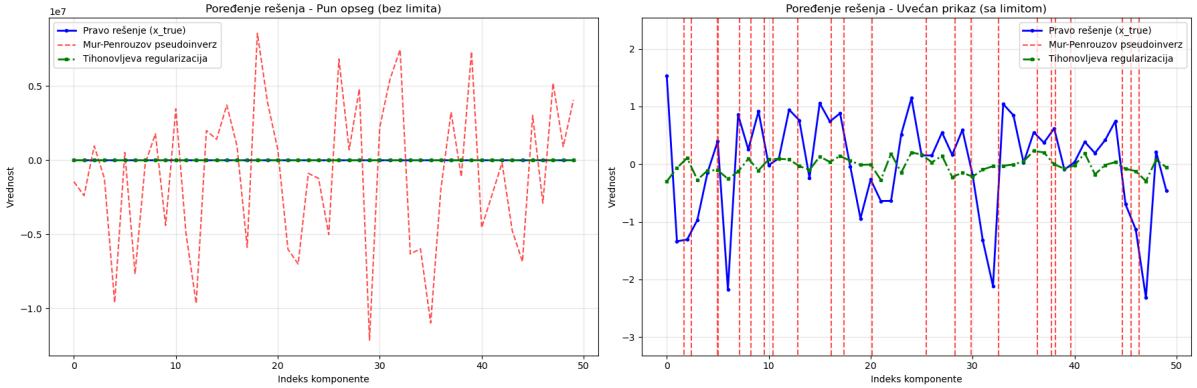
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))

# grafik 1: bez limita
ax1.plot(x_true, 'b-', linewidth=2, label='Pravo rešenje (x_true)', mark
ax1.plot(x_mp, 'r--', linewidth=1.5, label='Mur-Penrouzov pseudoinverz',
ax1.plot(x_reg, 'g-.', linewidth=2, label='Tihonovljeva regularizacija',
ax1.set_xlabel('Indeks komponente')
ax1.set_ylabel('Vrednost')
ax1.set_title('Poređenje rešenja - Pun opseg (bez limita)')
ax1.legend()
ax1.grid(True, alpha=0.3)

# grafik 2: sa limitom (fokus na x_true i x_reg)
ax2.plot(x_true, 'b-', linewidth=2, label='Pravo rešenje (x_true)', mark
ax2.plot(x_mp, 'r--', linewidth=1.5, label='Mur-Penrouzov pseudoinverz',
ax2.plot(x_reg, 'g-.', linewidth=2, label='Tihonovljeva regularizacija',
ax2.set_xlabel('Indeks komponente')
ax2.set_ylabel('Vrednost')
ax2.set_title('Poređenje rešenja - Uvećan prikaz (sa limitom)')
ax2.legend()
ax2.grid(True, alpha=0.3)
y_min = min(x_true.min(), x_reg.min()) - 1
y_max = max(x_true.max(), x_reg.max()) + 1
ax2.set_ylim(y_min, y_max)

plt.tight_layout()
plt.show()
```

```
Euklidska greška Mur-Penrouzovog pseudoinverza: 36076296.749
Euklidska greška Tihonovljeve regularizacije: 5.963
```



Pseudoinverz K^+ "invertuje" matricu, ali ne rešava problem loše uslovljenosti. Iako on daje matematički korektno rešenje, on ne rešava problem nestabilnosti. Ako K ima sopstvene vrednosti koje su veoma male ili jednake nuli, dešava se efekat koji je opisan u ranijim primerima, odnosno deljenje brojevima bliskim nuli, što dovodi do ogromnih faktora koji pojačavaju šum. To znači da i minimalan šum u podacima y postaje višestruko pojačan.

Sa druge strane, primena Tihonovljeve regularizacije omogućava da se ne desi "eksplozija" rešenja, što je takođe opisano ranije, a to možemo da potvrdimo na graficima iznad.

10/10поена

Задатак 3. У претходном задатку смо анализирали апстрактну матрицу K. У обради сигнала и слика, оператор замућења (blur) се често моделује као конволуција, која се у линеарној алгебри представља преко Теплицове матрице. Ово је типичан пример инверзног проблема: желимо да из замућеног и зашумљеног сигнала y повратимо оригинални оштар сигнал x.

Дат је систем

$$y = Ax + \varepsilon,$$

где је A матрица замућења, а ε адитивни шум. Користимо регуларизацију да пронађемо приближно решење оптимизационог проблема:

$$\min_x \ \|Ax - y\|_2^2 + \lambda \|x\|_2^2.$$

У машинском учењу, члан $\lambda\|x\|_2^2$ називамо L_2 регуларизацијом која спречава преприлагођавање (overfitting) тако што кажњава превелике вредности норме вектора x.

Проблем минимизације збира два квадрата $\|Ax - y\|_2^2 + \lambda\|x\|_2^2$ можемо записати као јединствен проблем најмањих квадрата коришћењем проширених матрица. Дефинишимо проширену матрицу $\tilde{A}$ и проширени вектор $\tilde{y}$ на следећи начин: $\tilde{A} = \begin{bmatrix} A \\ \lambda I \end{bmatrix}, \quad \tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}.$

a) Показати да важи следећа једнакост: $\|\tilde{A}x - \tilde{y}\|_2^2 = \|Ax - y\|_2^2 + \lambda\|x\|_2^2.$

(5 поена)

Greška u zadatku - da bi jednakost važila, proširena matrica mora biti:

$$\tilde{A} = \begin{bmatrix} A \\ \sqrt{\lambda} I \end{bmatrix}, \quad \tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}$$

DOKAZ:

Najpre računamo $\tilde{A}x - \tilde{y}$:

$$\tilde{A}x - \tilde{y} = \begin{bmatrix} A \\ \sqrt{\lambda}I \end{bmatrix} x - \begin{bmatrix} y \\ 0 \end{bmatrix}$$

$$= \begin{bmatrix} Ax \\ \sqrt{\lambda}Ix \end{bmatrix} - \begin{bmatrix} y \\ 0 \end{bmatrix}$$

$$= \begin{bmatrix} Ax - y \\ \sqrt{\lambda}x \end{bmatrix}$$

Sada računamo kvadrat Euklidove norme:

$$\|\tilde{A}x - \tilde{y}\|_2^2 = \left\| \begin{bmatrix} Ax - y \\ \sqrt{\lambda}x \end{bmatrix} \right\|_2^2$$

Kvadrat norme blok vektora jednak je zbiru kvadrata normi blokova:

$$= \|Ax - y\|_2^2 + \|\sqrt{\lambda}x\|_2^2$$

Koristimo osobinu norme: $\|c \cdot v\|_2 = |c| \cdot \|v\|_2$:

$$= \|Ax - y\|_2^2 + (\sqrt{\lambda})^2 \|x\|_2^2$$

$$\|Ax - y\|_2^2 + \lambda\|x\|_2^2$$

5/5поена

б) Из линеарне регресије знамо да проблем најмањих квадрата $\min_x \|\tilde{A}x - \tilde{y}\|_2^2$ има решење дефинисано нормалним једначинама $\tilde{A}^T\tilde{A}x = \tilde{A}^T\tilde{y}$.

Израчунати производе блока матрица $\tilde{A}^T\tilde{A}$ и $\tilde{A}^T\tilde{y}$ и доказати да се добија систем: $(A^TA + \lambda I)x = A^Ty$.

(5 поена)

Proširene matrice iz prethodnog koraka:

$$\tilde{A} = \begin{bmatrix} A \\ \sqrt{\lambda}I \end{bmatrix}, \quad \tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}$$

Izračunavanje $\tilde{A}^T\tilde{A}$:

$$\tilde{A}^T\tilde{A} = \begin{bmatrix} A^T & (\sqrt{\lambda}I)^T \end{bmatrix} \begin{bmatrix} A \\ \sqrt{\lambda}I \end{bmatrix}$$

Množenjem blokova dobijamo:

$$\tilde{A}^T\tilde{A} = A^TA + (\sqrt{\lambda}I)^T(\sqrt{\lambda}I)$$

Pošto je

$$(\sqrt{\lambda}I)^T(\sqrt{\lambda}I) = \lambda I,$$

sledi:

$$\tilde{A}^T \tilde{A} = A^T A + \lambda I$$

Izračunavanje $\tilde{A}^T \tilde{y}$:

$$\tilde{A}^T \tilde{y} = \begin{bmatrix} A^T & (\sqrt{\lambda} I)^T \end{bmatrix} \begin{bmatrix} y \\ 0 \end{bmatrix}$$

Dobijamo:

$$\tilde{A}^T \tilde{y} = A^T y + (\sqrt{\lambda} I)^T \cdot 0 = A^T y$$

Konačan sistem:

$$(A^T A + \lambda I)x = A^T y$$

5/5поена

в) Активирати код за симулацију замућења.

In [63]:
```
# 1. Генерисање оригиналног сигнала x (оштри скокови)
n = 100
x_true = np.zeros(n)
x_true[20:40] = 1.0
x_true[60:80] = 0.5

# 2. Формирање матрице замућења A (Gaussian blur)
c = np.exp(-np.linspace(0, 10, 10)**2 / 2) # Гаусово језгро
column = np.zeros(n)
column[:len(c)] = c
A = toeplitz(column)

# 3. Добијање замућеног сигнала y уз додатак малог шума
y_pure = A @ x_true
noise = np.random.normal(0, 0.05, n)
y = y_pure + noise
```

ВАШ ЗАДАТАК: реализовати рестаурацију

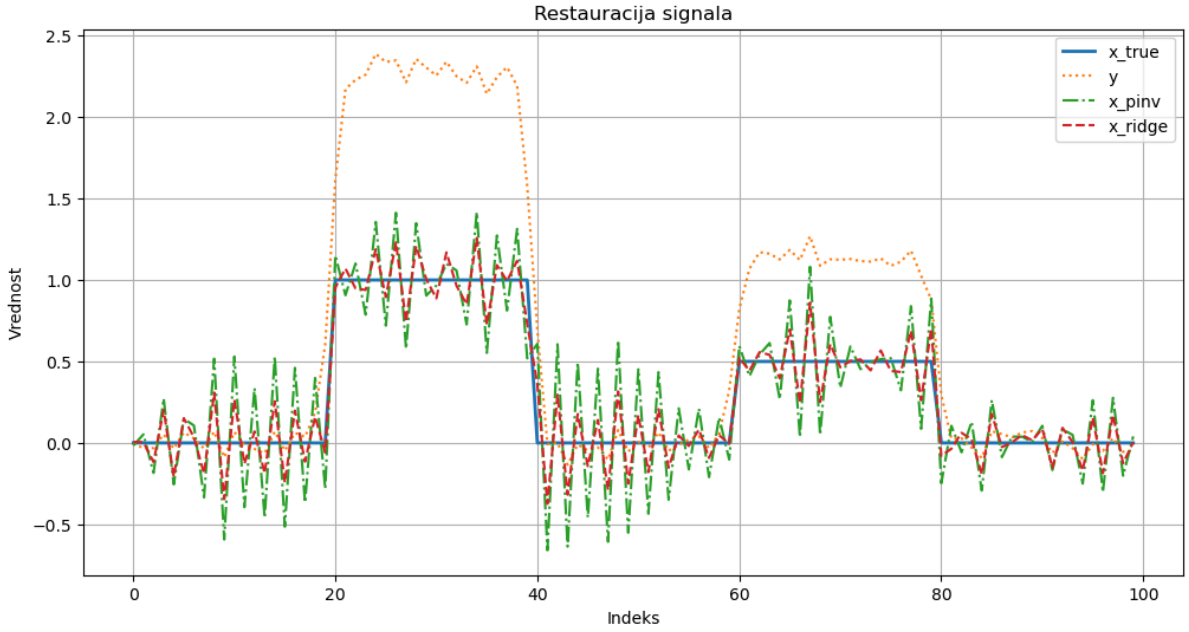
1. Решити систем (A.T @ A + lmbda * I)x = A.T @ y за lmbda = 0.01
2. Израчунати решење преко псеудоинверза x_pinv = np.linalg.pinv(A) @ y
3. На истом графику приказати x_true, y, x_pinv и x_ridge (рестаурисан сигнал)

(15 поена)

In [64]:
```
lmbda = 0.01
I = np.eye(n)
x_ridge = np.linalg.solve(A.T @ A + lmbda * I, A.T @ y)

x_pinv = np.linalg.pinv(A) @ y

plt.figure(figsize=(12,6))
plt.plot(x_true, label="x_true", linewidth=2)
plt.plot(y, label="y", linestyle=":")
plt.plot(x_pinv, label="x_pinv", linestyle="-.")
plt.plot(x_ridge, label="x_ridge", linestyle="--")
plt.legend()
plt.title("Restauracija signala")
plt.xlabel("Indeks")
plt.ylabel("Vrednost")
plt.grid(True)
plt.show()
```



15/15поена

г) На основу визуелног резултата, објасните зашто кажемо да регуларизација уводи пристрасност (bias) како би смањила варијансу (noise amplification). Како изгледа сигнал `x_ridge` у поређењу са `x_true` на местима где су скокови оштри?

(5 поена)

Na dobijenom grafiku jasno vidimo kompromis između pristrasnosti i varijanse (*bias-variance tradeoff*):

- Visoka varijansa (zelena linija - `x_pinv`): Rešenje preko pseudoinverza pokušava da savršeno reši sistem $Ax = y$. Pošto signal y (narandžasta tačkasta linija) poseduje šum, pseudoinverz taj

šum pojačava. Rezultat je zelena linija koja puno osciluje. Ovo rešenje je nepristrasno (ne postoji namerna greška modela), ali ima ogromnu varijansu zbog osetljivosti na šum.
- Korišćenjem regularizacije, uvodimo pristrasnost radi stabilnosti (crvena linija - `x_ridge`): Algoritam sada "kažnjava" prevelike komponente vektora x, što direktno prigušuje oscilacije. Kao što vidimo, crvena linija je znatno mirnija i bliža originalnom signalu (pogotovo na segmentima 40-60 i 80-100) od zelene. Žrtvujemo savršeno (ali nestabilno) matematičko rešenje uvođenjem blage greške (*bias*), kako bismo drastično smanjili osetljivost na šum (varijansu) i dobili fizički smisleniji rezultat.

Kada uporedimo `x_ridge` (crvena isprekidana linija) sa originalnim signalom `x_true` (plava puna linija) na mestima gde se dešavaju oštri skokovi (npr. indeksi 20, 40, 60 i 80), primećujemo da originalni signal ima skoro savršeno vertikalan prelaz, dok regularizovani signal `x_ridge` ne može da isprati ovaj trenutni skok već je više "zaobljen".

5/5поена

Задатак 4. СВД декомпозиција матрице $A \in M_{m\times n}$ ранга r гласи $A = U\Sigma V^T$.

Матрицу A можемо записати и као суму r матрица ранга 1 (спољашњих производа): $A = \sum_{i=1}^{r} \sigma_i u_i v_i^T$.

Овај приказ нам омогућава да направимо апроксимацију ниског ранга A_k тако што задржимо само првих k највећих сингуларних вредности: $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T, \quad k < r.$

а) На основу Екарт–Јанг–Мирски теореме, апроксимација A_k је најбоља апроксимација ранга k у смислу Фробенијусове норме.

Питање: Колико износи квадрат грешке апроксимације $\|A - A_k\|_F^2$ изражен преко одбачених сингуларних вредности?

(5 поена)

Polazimo od definicije matrice A i njene aproksimacije A_k ranga k (gde je $k < r$):

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^T$$

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$$

Matrica greške aproksimacije predstavlja razliku ove dve matrice. Pošto A_k sadrži prvih k članova sume matrice A, kada ih oduzmemo, ostaju samo preostali, odbačeni članovi (od indeksa $k+1$ do r):

$$A - A_k = \sum_{i=k+1}^{r} \sigma_i u_i v_i^T$$

S obzirom na to da su levi i desni singularni vektori (u_i i v_i) međusobno ortonormirani, izraz iznad zapravo predstavlja SVD dekompoziciju same matrice greške. Njene singularne vrednosti su upravo te odbačene singularne vrednosti originalne matrice A: $\sigma_{k+1}, \sigma_{k+2}, \ldots, \sigma_r$.

Sada je potrebno da izračunamo normu matrice $A - A_k$. Jedno od osnovnih svojstava Frobenijusove norme matrice jeste da je njen kvadrat jednak sumi kvadrata svih njenih singularnih vrednosti [1] (https://math.stackexchange.com/questions/1142565/how-to-express-the-frobenius-norm-of-a-matrix-as-the-squared-norm-of-its-singula).

Prema tome, dobijamo sledeći izraz:

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$$

5/5поена

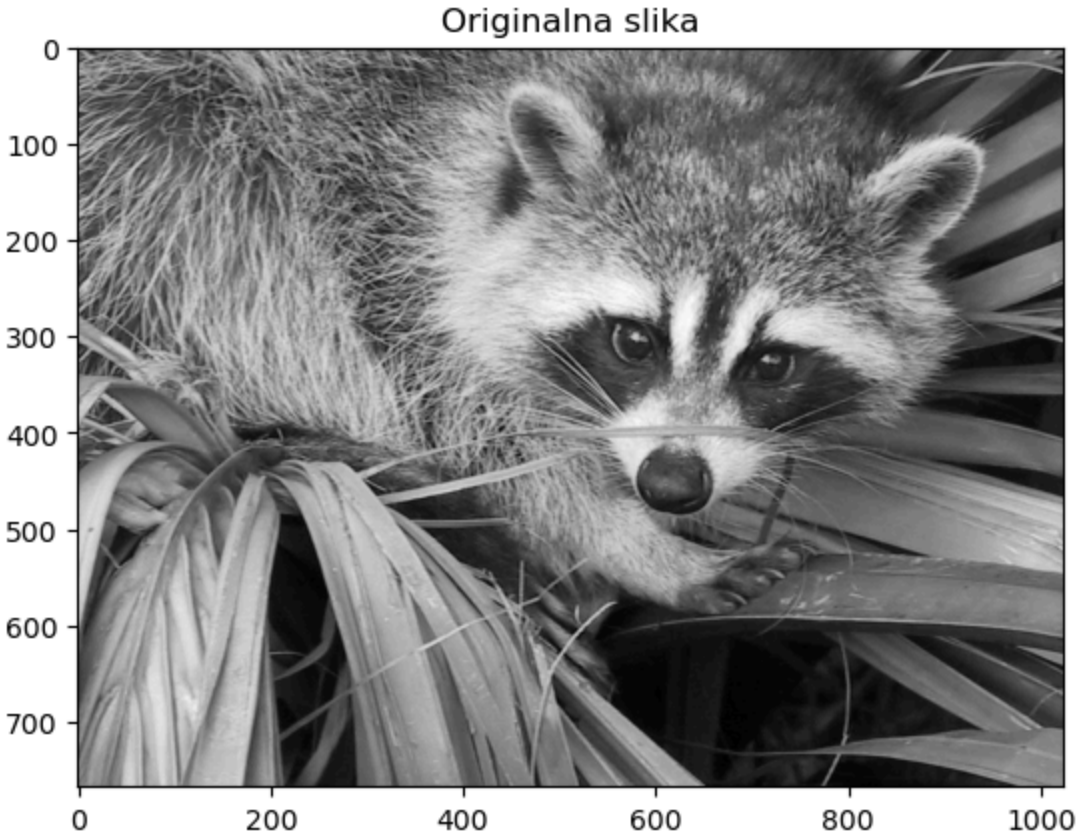
б) У AI апликацијама често меримо проценат „сачуване информације“ (енергије) преко израза

$$E(k) = \frac{\|A_k\|_F^2}{\|A\|_F^2} = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

Ако желимо да компресујемо матрицу A тако да сачувамо **95% информација**, потребно је изабрати најмањи број компоненти k који задовољава услов?

In [65]:
```
import scipy.datasets as scd
```

In [66]:
```
rakun=scd.face()
sk_vredn=rakun/255
A = sk_vredn @ [0.2126, 0.7152, 0.0722]
A.shape
plt.imshow(A, cmap="gray");
plt.title("Originalna slika");
```



ВАШ ЗАДАТАК:

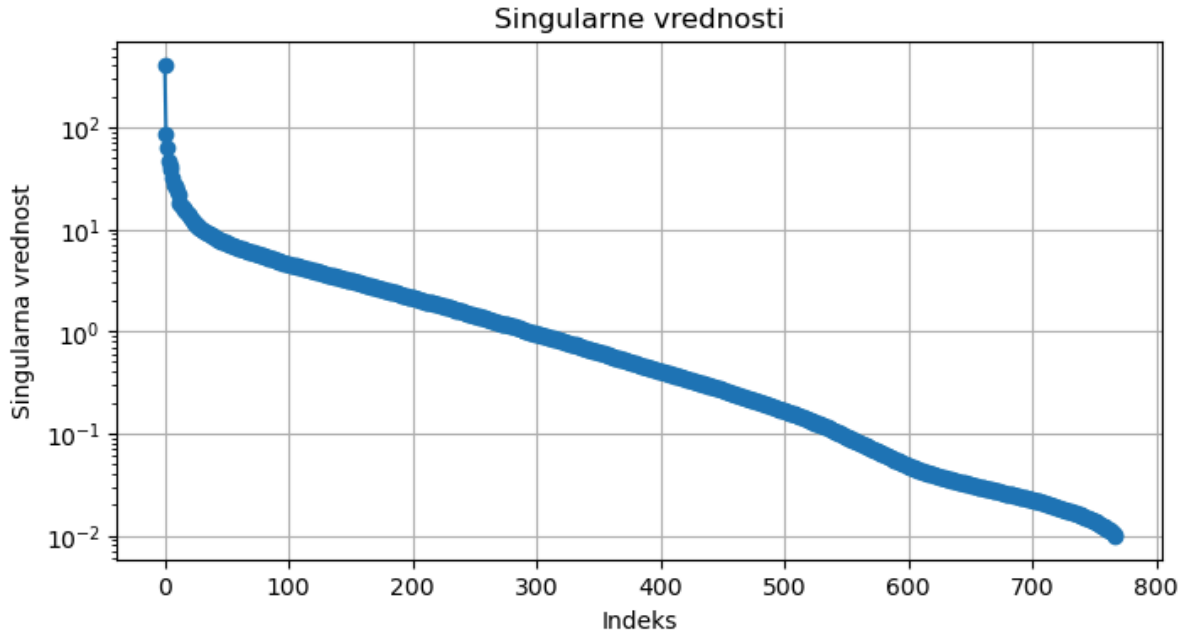
1. Нацртати график сингуларних вредности (S) на логаритамској скали. Шта нам овај график говори о могућности компресије ове слике?
2. Нацртати график кумулативне енергије E(k) у функцији од k. Пронаћи најмање k за које је E(k) > 0.95 (95% енергије).
3. Реконструисати слику A_k за k = 5, 20, 50 и за k који даје 95% енергије.
4. Израчунати однос компресије за свако k.
   - Оригинал број података: m * n
   - Компресован број података: k * (m + n + 1)
   - Ratio = (m * n) / (k * (m + n + 1))
5. Приказати оригинале и реконструисане слике једу поред друге.

(5 поена)

In [67]:
```
m, n = A.shape

U, S, Vt = svd(A, full_matrices=False)

# grafik singularnih vrednosti na log skali
plt.figure(figsize=(8,4))
plt.semilogy(S, 'o-')
plt.title("Singularne vrednosti")
plt.xlabel("Indeks")
plt.ylabel("Singularna vrednost")
plt.grid(True)
plt.show()
```

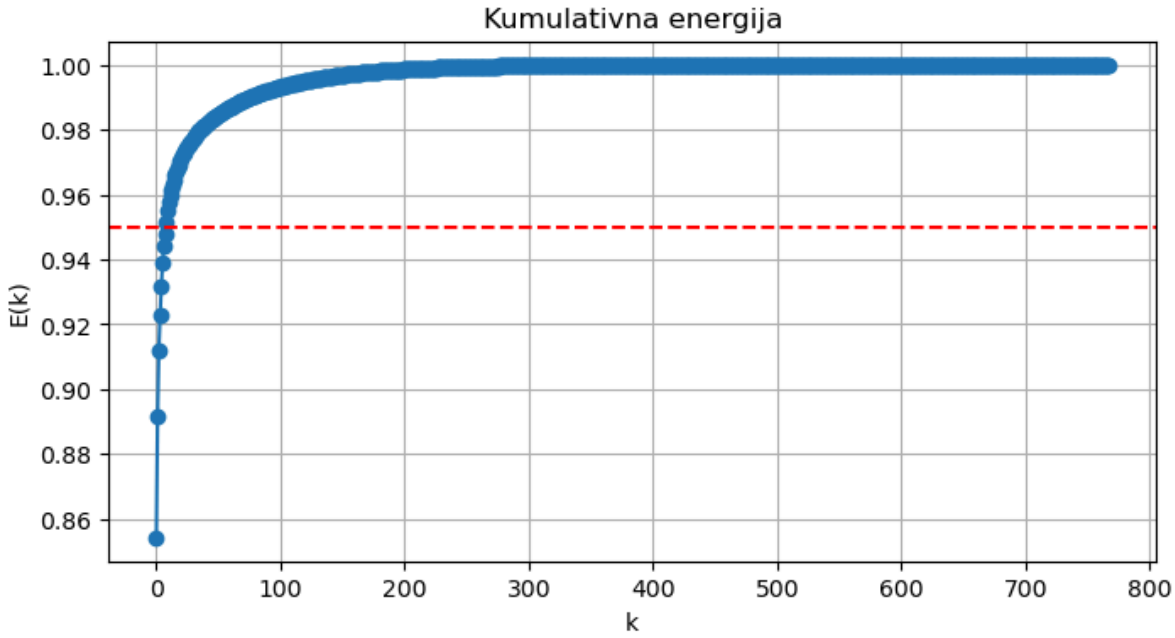


Vidimo da singularne vrednosti brzo opadaju (eksponencijalno, što se bolje vidi na običnom, a ne logaritamskom grafiku), što znači da je većina energije koncentrisana u prvih nekoliko komponenti, što je dobro za kompresiju jer ćemo moći sliku lepo da predstavimo sa dosta manje memorijskog prostora.

In [68]:

```
# grafik kumulativne energije
E = np.cumsum(S**2) / np.sum(S**2)
plt.figure(figsize=(8,4))
plt.plot(E, 'o-')
plt.axhline(0.95, color='r', linestyle='--')
plt.title("Kumulativna energija")
plt.xlabel("k")
plt.ylabel("E(k)")
plt.grid(True)
plt.show()

k95 = np.argmax(E >= 0.95) + 1
print("Najmanje k za 95% energije:", k95)
```




```
Najmanje k za 95% energije: 9
```

In [69]:

```
# rekonstrukcija
def reconstruct(U, S, Vt, k):
    return U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]

k_values = [5, 20, 50, k95]
reconstructions = [reconstruct(U, S, Vt, k) for k in k_values]

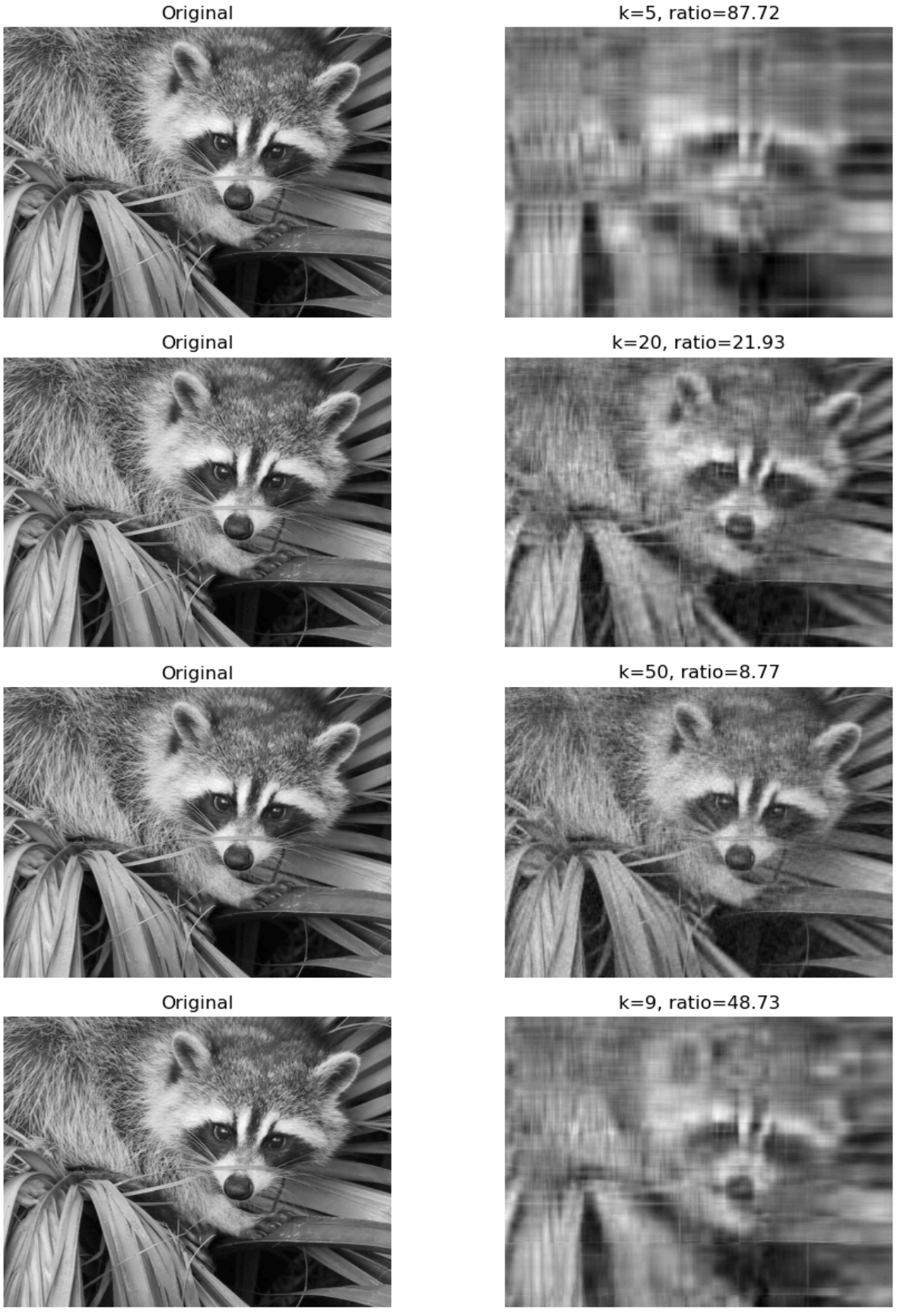
# računanje odnosa kompresije
ratios = {}
for k in k_values:
    original = m * n
    compressed = k * (m + n + 1)
    ratio = original / compressed
    ratios[k] = ratio
    print(f"k={k}, ratio={ratio:.2f}")

# prikaz
plt.figure(figsize=(10,12))

for i, k in enumerate(k_values):
    plt.subplot(len(k_values), 2, 2*i+1)
    plt.imshow(A, cmap="gray")
    plt.title("Original")
    plt.axis("off")
    plt.subplot(len(k_values), 2, 2*i+2)
    plt.imshow(reconstructions[i], cmap="gray")
    plt.title(f"k={k}, ratio={ratios[k]:.2f}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
k=5, ratio=87.72
k=20, ratio=21.93
k=50, ratio=8.77
k=9, ratio=48.73
```


Original
k=5, ratio=87.72
Original
k=20, ratio=21.93
Original
k=50, ratio=8.77
Original
k=9, ratio=48.73

5/5поена

в) Често мали пиксели (шум) утичу на најмање сингуларне вредности. Покушајте да додате случајан шум слици:

In [70]:
```
A_noisy = A + 0.1 * np.random.randn(*A.shape)
```

Примените СВД и реконструишите слику са $k = 30$.

In [71]:
```
U, S, Vt = svd(A_noisy, full_matrices=False)

k = 30
A30 = reconstruct(U, S, Vt, k)

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(A, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(A_noisy, cmap="gray")
plt.title("Sa šumom")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(A30, cmap="gray")
plt.title("Rekonstrukcija k=30")
plt.axis("off")

plt.tight_layout()
plt.show()
```



5/5поена

Објасните зашто СВД апроксимација ниског ранга може да послужи као метода за чишћење шума (denoising). Повежите ово са претходним задатком о регуларизацији и стабилности инверзних проблема.

(10 поена)

SVD aproksimacija niskog ranga funkcioniše kao metoda za denoising zato što razdvaja signal na komponente po značaju.

- Velike singularne vrednosti odgovaraju dominantnim strukturama u podacima (glavne konture i oblici na slici).
- Male singularne vrednosti često hvataju fine detalje, ali i slučajni šum. Kada dodamo slučajan šum slici, on se najviše manifestuje u tim najmanjim singularnim vrednostima. Ako zadržimo samo prvih k komponenti (najveće singularne vrednosti), dobijamo aproksimaciju koja čuva glavnu strukturu slike, dok se male singularne vrednosti (u kojima je šum najizraženiji) odbacuju. Rezultat je slika koja je „čistija“ i stabilnija.

To možemo da vidimo na primeru iznad. Slika sa šumom je "zrnasta", dok rekonstruisana slika za $k = 30$ ima manje detalja jer je zadržano samo 30 komponenti, ali je šum više eliminisan (na primer, crni deo na originalnoj slici ispod rakunove glave je na slici sa šumom više siv, a na rekonstruisanoj slici sa eliminisanim šumom je bliži crnoj boji, mada ima manje detalja).

Kod inverznih problema (npr. rekonstrukcija signala), mala sopstvena ili singularna vrednost dovodi do pojačanja šuma kada radimo direktnu inverziju. Regularizacija (Ridge / Tihonovljeva) uvodi pristrasnost tako što prigušuje doprinos tih malih vrednosti, čime se smanjuje varijansa i stabilizuje rešenje. SVD denoising primenjuje sličan princip - odbacuje male singularne vrednosti čime se uvodi pristrasnost (slika postaje malo „glatkija“), ali time drastično smanjujemo varijansu (šum se eliminiše).

10/10поена

Да ли би СВД компресија подједнако добро радила на слици која представља чисто бели шум (random noise)? Зашто? (Помоћ: Размислите о брзини опадања сингуларних вредности).

(10 поена)

SVD aproksimacija niskog ranga radi dobro kada singularne vrednosti matrice brzo opadaju. Kod prirodnih slika prvih nekoliko singularnih vrednosti sadrži većinu energije, dok su najmanje vrednosti male i mogu se odbaciti bez velikog gubitka informacija. Na taj način se uklanja šum, a čuva se struktura slike.

Kod čisto belog šuma situacija je drugačija. Matrica koja predstavlja random noise nema jasnu strukturu i njene singularne vrednosti ne opadaju brzo, već su raspoređene približno ravnomerno. To znači da nema dominantnih komponenti koje nose signal, pa odbacivanje bilo kog dela spektra vodi do gubitka informacije jednako koliko uklanja šum.

Dakle, SVD kompresija ne bi radila podjednako dobro na slici koja je čisti beli šum, jer se oslanja na brzi pad singularnih vrednosti, a kod belog šuma takav pad ne postoji.

To možemo da vidimo i pomoću sledećeg Python koda.

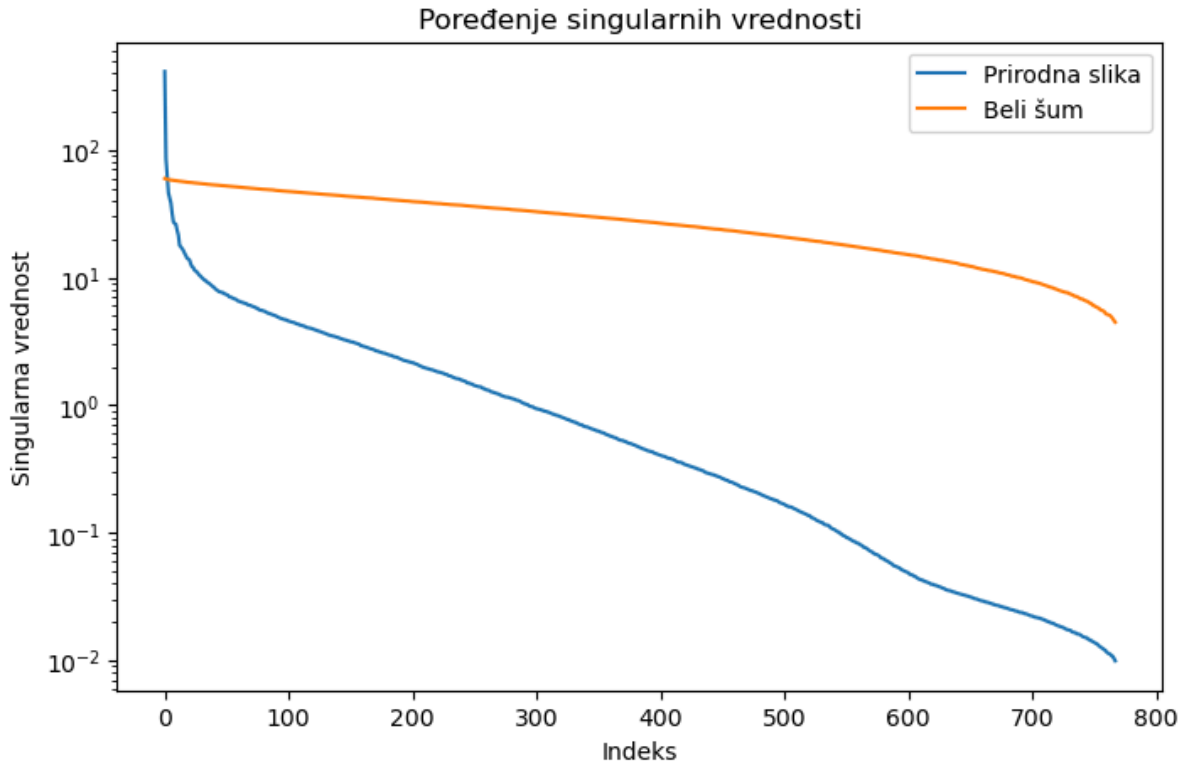
In [72]:
```
A_noise = np.random.randn(m, n)

U_img, S_img, Vt_img = np.linalg.svd(A, full_matrices=False)
U_noi, S_noi, Vt_noi = np.linalg.svd(A_noise, full_matrices=False)

plt.figure(figsize=(8,5))

plt.semilogy(S_img, label='Prirodna slika')
plt.semilogy(S_noi, label='Beli šum')

plt.title("Poređenje singularnih vrednosti")
plt.xlabel("Indeks")
plt.ylabel("Singularna vrednost")
plt.legend()
plt.show()
```



10/10поена

Укупно 105/105поена